Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #4

List operations

Agenda

- **Forward vs backward recursion**
  - sum
  - generics
  - Comparative analysis, pros and cons
- **append3**
  - Forms
  - Efficiency analysis and justification
  - Nondeterministic call
- **Delete from a list**
  - One, just one, all, repeating the query

Forward vs backward recursion

- Forward vs backward - alternative approaches to handle data
- Data is split into partitions
- **Forward** = processing takes place when the current item is processed when is first encountered, and the rest of the data (the other components than the item) are processed AFTER the current item is processed.
- **Backward** = starts by processing all next after current item (the other components than the item) via recursive call(s) while the current item is processed just AFTER we return from recursion(s).
  - *NOTE:* it is returned from recursive call (and **NOT from backtracking!!!**).
- In case of lists [H|T]:
  - *forward* handles H first and next T (via recursive call),
  - *backward* starts by solving the problem on T (via recursive call), and just when returning from the call H is processed.

Forward sum

```
% sum_fw/3 (List, Accumulator, Result).

sum_fw([],PartialSum,PartialSum).             % final result, copies the value of the partial
                                              % result with default unification

sum_fw([H|T],PartialSum,Sum):-
        NewPartialSum is PartialSum + H,      % do process the current item
        sum_fw(T,NewPartialSum,Sum).          % go ahead with the remaining struct
```

- List decomposed into [H|T]
  - Starts by processing (addition here) the current item (H here)
  - Continue with processing the rest of the structure (one recursive call here, as partition is H and T)
  - This implies the items are added in the sum forward (starting from the first one).
- ->->->...-> processing is performed in the order of items in the structure
- The accumulated result represents the sum of values from the beginning to the current item

Backward sum

```
% sum_bw/2 (List, Result).

sum_bw([],0).                      % result gets initialized. Empty input, null
sum_bw([H|T],Sum):-
        sum_bw(T,TailSum),         % call processing first on rest of the partition
        Sum is TailSum + H.        % do process the current item
```

- List decomposed into [H|T]
  - Starts with processing T, same predicate, recursive call
  - When returned, use the obtained result (TailSum) to evaluate the overall result on the whole list (Sum)
  - This implies the items are added in the sum backwards (starting from the last one).
- ---... --> first go this way (all way to the end) doing nothing (just call)
- <- the processing occurs after returning from the call, one at a time
  - in the opposite order, *last* item *first* processed, before last item second processed, ….
  - third, second and first items being last processed in this specific order (first is last)
- The accumulated result is the sum of values from the current item to the end

Forward vs backward sum

- Forward solution needs specific initial call
  - write a special predicate (a wrapper) to make it in a sound way (avoid the need of user knowing how to initialize the accumulator parameter):

```
% sum_fw/2 (List, Result)
sum_fw(List,Sum):-
        sum_fw(List,0,Sum).      % same partial result as in case of backward
                                 % nothing yet processed, null result.
                                 % same as in stop condition for bwd.
```

- If the input list is [1,2,3,4,5,6,7] what is going to be the partial result (PartialSum) and (TailSum) respectively on forward and backward solutions?
- Try to **estimate before running** them.
- Follow the textbook to update the predicate with the necessary lines to <u>print</u> the values.

```
% sum_bw/2 (List, Result).

sum_bw([],0).
sum_bw([H|T],Sum):-
        sum_bw(T,TailSum),
        Sum is TailSum + H.
```

```
% sum_fw/3 (List, Accumulator, Result).

sum_fw([],PartialSum,PartialSum).
sum_fw([H|T],PartialSum,Sum):-
        NewPartialSum is PartialSum + H,
        sum_fw(T,NewPartialSum,Sum).
```

Forward generic

```
% forward_recursion/3 (Input, PartialResult, FinalResult)
```

% final result (arg 3) copies partial result(arg 2) value with default unification

```
forward_recursion([],Result,Result).
```

% partition data, split into H and T

```
forward_recursion([H|T],PartialResult,Result):-
```

% start by processing the current item and thus

% updating previous PartialResult to NewPartialResult via processing *do/3*

```
        do(PartialResult,H,NewPartialResult),
```

% process the rest of the structure with recursive call.

```
        forward_recursion(T,NewPartialResult,Result).
```

% forward_recursion_call/2 (Input, Output)

% make the initialization with a separate predicate (wrapper) to avoid mandatory user initialization

```
forward_recursion_call(Input,Output):-
        forward_recursion(Input,InitialValueOfResult,Output)
```

Backward generic

```
% backward_recursion/2 (Input, Output)
```

% empty input, make initialization backwards

```
backward_recursion([],InitialValueOfResult).
```

```
backward_recursion([H|T],NewPartialResult):-
```

% starts with processing the rest of the structure;

% all partition but the current item.

```
        backward_recursion(T,PartialResult),
```

%process the current item

```
        do(PartialResult,H,NewPartialResult).
```

- No need for a specific initial call, hence, no wrapper predicate.

Forward vs backward recursion

```
fw_rec([],R,R).
fw_rec([H|T],PR,R):-
      do(PR,H,NPR),
      fw_rec(T,NPR,R).

fw_rec(Input,Output):-
      fw_rec(Input,InitialValueOfResult,Output)



bw_rec([],InitialValueOfResult).
bw_rec([H|T],NPR):-
      bw_rec(T,PR),
      do(PR,H,NPR).
```

Forward vs backward pros and cons

	Forward	Backward
+	• Process "as we go" => structure is processed from front to end => intermediate results could be useful (the result of the structure "so far"). • In concurrent processing that result is made available and another process using it can start immediately • Last Call Optimization = reusing the same stack area without the need of restoring it back	• Needs no initialization => needs no specific call => needs no wrapper => needs no additional argument
-	• Needs specific initial call => always make a wrapper to initialize the accumulator • Needs additional argument (partial result)	• Intermediate results are seldom useful • The result of the structure is known just at the end of processing, so, concurrency is postponed on sync

Concatenate 3 lists

- Use what you ***have*** vs use what you ***know***
- We have the concatenation of 2 lists
- Use it twice.

```
append([],List,List).
append([Head|Tail],List,[Head|Rest]):-
        append(Tail,List,Rest).
```

- To put together L1, L2 and L3 do the following
  - (L1+L2) + L3 OR L1 + (L2+L3)

Concatenate 3 lists: Use what you have 1

```
append([],L,L).
append([H|T],L,[H|R]):-
        append(T,L,R).
```

- (L1+L2) + L3; say |L1|=n1, |L2|=n2, |L3|=n3

```
append3_1(L1,L2,L3,Result):
        append(L1,L2,Intermediate),          % link L2 at the end of L1 = Intermediate
        append(Intermediate,L3,Result).      % link L3 at the end of the intermediate result
```

- Efficiency:
  - O(n1) for the 1st call (links L2 at the end of L1)
  - O(n1+n2) for the 2nd call, length Intermediate of is length of L1 and L2 (links L3 at the end of Intermediate)
  - Overall: t(n)=2n1+n2

Concatenate 3 lists: Use what you have 2

```
append([],L,L).
append([H|T],L,[H|R]):-
        append(T,L,R).
```

- L1 + (L2+ L3); say |L1|=n1, |L2|=n2, |L3|=n3

```
append3_2(L1,L2,L3,Result):-
        append(L2,L3,Intermediate),          % link L3 at the end of L2 = Intermediate
        append(L1,Intermediate,Result).      % link intermediate at the end of the first list L1
```

- Efficiency:
  - O(n2) for the 1st call, decomposes L2 (links L3 at the end of L2)
  - O(n1) for the 2nd call, decomposes L1 (links Intermediate at the end of L1)
  - Overall: t(n)=n1+n2
- Observations:
  - **Second version better regardless of the input!**
  - Order of calls matters (again!)
  - How can we use this in a standalone predicate?

Concatenate 3 lists: Use what you know

- What do we know?
  - Concatenation via decomposition of the first argument! Use it!

```
append3_3([H|T],L2,L3,[H|R]):-
        % as long as the first argument is nonempty, decompose it
        append3_3(T,L2,L3,R).
append3_3([],[H|T],L,[H|R]):-
        % once first argument becomes empty, you are back on 2 list concatenation
        append3_3([],T,L,R).
append3_3([],[],L,L).
```

- Observations:
  - Clauses 2 and 3 are disjoint from clause 1 (indexation on the first argument would treat them separately, i.e. cl1 one entrance, cl 2 and 3 another one). Therefore, it does NOT matter where clause 1 is placed
  - On the other hand, clause 3 should come AFTER clause 2 (as indexation on the second argument is NOT available

Concatenate 3 lists: Use what you know – contd.

```
append3_3([H|T],L2,L3,[H|R]):-
        append3_3(T,L2,L3,R).          % decomposes first list
append3_3([],[H|T],L,[H|R]):-
        append3_3([],T,L,R).           % decomposes second list
append3_3([],[],L,L).
```

- Observations:
  - How is it done? (resembles behavior of version1)
    - Decomposes `L1` to traverse it and adds each item in result (behaves as version 1, as in "link `L2` at the end of `L1`")
    - Decomposes `L2` to traverse it and adds each item in result (links `L3` at the end of `L1` concatenated to `L2`).
  - Efficiency (resembles in performance to version _2): t(n)=n1+n2
    - O(n1) first clause
    - O(n2) second clause

```
append3_2(L1,L2,L3,Result):-
        append(L2,L3,Intermediate),       % link L3 at the end of L2 = Intermediate
        append(L1,Intermediate,Result).   % link intermediate at the end of the first list L1
```

Concatenate 3 lists: Use what you know – contd.

- Behavior: resembles version_1 (L1+L2) + L3
- Efficiency: resembles version_2 t(n)=n1+n2
- How is possible? Behavior V1 and performance V2?
- Explain!
- Explain which of the 3 versions is best?
- Which to use and why?
- Which should never be used? Why?

Concatenate 3 lists: Nondeterministic call

- Given `append3` predicate and the call: `?-append3(X,Y,Z,[1,2]).`
- What is the meaning of the call?
  - Non-deterministic call
  - Which lists X,Y and Z concatenated form list [1,2].
- What is/are the result/s?

X	Y	Z
[]	[]	[1,2]
[]	[1]	[2]
[]	[1,2]	[]
[1]	[]	[2]
[1]	[2]	[]
[1,2]	[]	[]

- Other results? Why not?
- Which order/why?
  - Depends on the implementation.
  - Identify (BEFORE running) the order of results in each implementation
  - For `append3` with `append`, the `append` order of clauses is MANDATORY (fact first), otherwise it enters infinite loop with 3 free variables on the first call.

Delete from a list

- Given a list, remove one item from it.
- How many arguments/why?

```
% delete/3 (Item_to_remove, Input_list, Output_list)

delete(X,[X|T],T).     % if item to remove = head of input, don't put it in output
delete(X,[H|T],[H|R]):-      % otherwise, keep it on output and
       delete(X,T,R).        % remove from tail
```

- What are the results on this query when the item occurs several times?

```
?- delete(4,[1,4,2,4,3,4],R).
R=[1,  2,4,3,4] ;                  % Next
R=[1,4,2,  3,4] ;                  % Next
R=[1,4,2,4,3  ] ;                  % Next
false
```

- What happens if the item is not present in the list?

```
?- delete(5,[1,4,2,4,3,4],R).
false
```

Delete from a list

```
delete(H,[H|T],T).
delete(X,[H|T],[H|R]):-
        delete(X,T,R).
```

So, the meaning of the predicate is:

- **delete exactly one occurrence** of an item from the list.

- What are the results on this query? Why?

```
?- delete(1,[1,2,1,3,1],R).
R = [  2,1,3,1]  ;          % Next
R = [1,2,  3,1]  ;          % Next
R = [1,2,1,3  ]  ;          % Next
false.
```

- What about this query? Why?

```
?- delete(4,[1,2,1,3,1],R).
false.
```

Delete from a list

```
delete(X,[X|T],T).    % if item to remove = head of input, don't put it in output
delete(X,[H|T],[H|R]):-       % otherwise, keep it on output and
        delete(X,T,R).        % remove from tail
delete(_,[],[]).
% when the empty list is reached,
% whatever element is assumed to
% be deleted, done, result empty
```

- What are the results on call when the item occurs several times?
- What happens in case the item is not present in the list?

Delete from a list

```
delete(X,[X|T],T).
delete(X,[H|T],[H|R]):-
       delete(X,T,R).
delete(_,[],[]).
```

So, the meaning of the predicate is:

- **delete one occurrence** of an item from the list;
- if **absent, do NOTHING** (leave the list unchanged).

- What are the results on call?

```
?- delete(1,[1,2,1,3,1],R).
R = [  2,1,3,1] ;              % Next
R = [1,2,  3,1] ;              % Next
R = [1,2,1,3  ] ;              % Next, what's next? Why?
R = [1,2,1,3,1] ;
false.
```

- What about the call? Why?

```
?- delete(4,[1,2,1,3,1],R).
R = [1,2,1,3,1] ;              % Next
false.
```

Delete from a list

- What are the differences when the 2 implementations (without/with 3rd clause) are compared? Explain!
- What happens if the clause when item is found cuts the backtrack?
- Implementation without 3rd clause:

```
delete(X,[X|T],T):-!.
delete(X,[H|T],[H|R]):-
      delete(X,T,R).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
  - p:-**q,r**,!,s.
  - The cut implies a “default” negation of **q,r** (as **not(q,r)**) in all clauses after it.
- In the predicate `delete`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete from a list

```
delete(X,[X|T],T):- !.        % means: delete(X,[H|T],T):-X=H,!,
delete(X,[H|T],[H|R]):-  % therefore, an implicit X\=H is here
        delete(X,T,R).

?- delete(1,[1,2,1,3,1],R).
R=[2, 1, 3, 1] ;  % Why? Next?
?- delete(4,[1,2,1,3,1],R2).
false.
```

- In the implementation with 3rd clause:

```
delete(X,[X|T],T):- !.
delete(X,[H|T],[H|R]):-
        delete(X,T,R).
delete(_,[],[]).

?- delete(1,[1,2,1,3,1],R).
R=[2, 1, 3, 1] ;  % Why? Next?
?- delete(4,[1,2,1,3,1],R2).
R=[1,2,1,3,1];  % Why? Next?
```

Delete all occurrences of an item from a list

Start from the first version of the predicate

```
delete(X,[X|T],T).
delete(X,[H|T],[H|R]):-
        delete(X,T,R).
```

```
delete(X,[X|T],R):-  % if item to remove = head of input, don’t put it in output
        delete(X,T,R).  % but also remove other occurrences from tail
delete(X,[H|T],[H|R]):-     % otherwise, keep it on output and
        delete(X,T,R).       % remove from tail
```

- Could it be just this?
- Justify!

Delete all occurrences of an item from a list

% if the item to remove is head of input, don't put it on output

```
delete_all(X,[X|T],R):-
       delete_all(X,T,R).   % but also remove other occurrences from tail
% otherwise, keep it on output and
delete_all(X,[H|T],[H|R]):-
       delete_all(X,T,R).   % remove from tail
delete_all(_,[],[]).        % when [ ] is reached, done, result empty
```

```
?- delete_all(1,[1,2,1,3,1],R).
R = [2,3]
```

- What happens if we repeat the query? Why?
- How many answers? Which order? Explain!
- How can we obtain just the answer from above?

```
R = [2, 3, 1]
R = [2, 1, 3]
R = [2, 1, 3, 1]
R = [1, 2, 3]
R = [1, 2, 3, 1]
R = [1, 2, 1, 3]
R = [1, 2, 1, 3, 1]
```

Delete all occurrences of an item from a list

```
delete_all(X,[X|T],R):-
     !,
     delete_all(X,T,R).
delete_all(X,[H|T],[H|R]):-
     delete_all(X,T,R).
delete_all(_,[],[]).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
  - p:-**q,r**,!,s.
  - The cut implies a "default" negation of **q,r** (as **not(q,r)**) in all clauses after it.
- In the predicate `delete_all`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete all occurrences of an item from a list

```
delete_all(X,[X|T],R):-!,
      delete_all(X,T,R).
```

What does ! negate?
Is the default unification! The clause above is like:

```
delete_all(X,[H|T],R):-
      X=H, !,
      delete_all(X,T,R).
```

Therefore, the cut negates **X=H**, thus, in the next clauses, there is a default **X\=H**.

Delete all occurrences of an item from a list

```
delete_all(X,[X|T],R):-
     !,
     delete_all(X,T,R).
delete_all(X,[H|T],[H|R]):-
     delete_all(X,T,R).
delete_all(_,[],[]).


?- delete_all(1,[1,2,1,3,1],R).
R=[2,3]
```

- What is the result? Why?
- What happens if we repeat the query? Why? How many answers? Which order? Explain!

Conclusion

- Order of clauses matters
- Order of calls matters
- Always estimate performance
- Meaning of
  - Repeating the query
  - The cut
- Questions?